Report (Assignment 3)

Theory Questions

1. Concept of Soft Prompts: How does the introduction of ”soft prompts” address the limitations of discrete text prompts in large language models? Why might soft prompts be considered a more flexible and efficient approach for task-specific conditioning?

The introduction of "soft prompts" in large language models addresses limitations inherent in discrete, text-based prompts by leveraging continuous vectors in embedding space rather than rigid text inputs.

Soft prompts are composed of trainable embeddings (essentially learned weights) rather than fixed text tokens, allowing them to capture nuanced and abstract representations that might be challenging or inefficient to convey with text alone. By training on a small set of parameters specific to the task, soft prompts allows for parameter-efficient approach which enables faster task-specific conditioning while reducing computational overhead and memory requirements compared to traditional fine-tuning, which updates the entire model.

2. Scaling and Efficiency in Prompt Tuning: How does the efficiency of prompt tuning relate to the scale of the language model? Discuss the implications of this relationship for future developments in large-scale language models and their adaptability to specific tasks.

As language models scale up, fine-tuning all parameters for specific tasks becomes increasingly resource-intensive and computationally demanding. Prompt tuning, however, modifies only a small set of parameters (such as a few continuous embeddings), reducing memory and compute requirements. Large-scale language models often contain richer representations, making it possible to elicit complex task-specific behaviors by tuning only the prompt. Thus, prompt tuning allows larger models to perform task adaptation without the burden of extensive parameter updates.

Larger models possess a greater capacity for learning diverse patterns and abstractions, allowing them to generalize more effectively across different tasks with minimal task-specific input. In prompt tuning, since only the prompt needs to be tuned, the model can rapidly adapt to new domains or instructions, enhancing scalability and adaptability in large-scale deployments. In the future, prompt tuning’s efficiency in large-scale models suggests that large language models can become

increasingly modular, adaptable, and cost-effective for diverse, task-specific applications.

3. Understanding LoRA: What are the key principles behind Low-Rank Adaptation (LoRA) in fine-tuning large language models? How does LoRA improve upon traditional fine-tuning methods regarding efficiency and performance?

Low-Rank Adaptation (LoRA) is a technique designed to make fine-tuning large language models more efficient and parameter-efficient. It works by applying the following principles:

Principle of Low-Rank Decomposition: Instead of updating the entire weight matrices, LoRA injects a low-rank decomposition (usually small matrices) that, when multiplied, approximate the task-specific adaptations. This way, LoRA isolates and applies only the necessary task-specific changes without overloading the model with unnecessary parameter updates.

Adding Trainable Layers as Low-Rank Matrices: During fine-tuning, LoRA introduces additional trainable low-rank matrices into the model's architecture that are added to the frozen, pre-trained weights. These low-rank matrices adjust the output of specific layers to match the new task requirements without modifying the original parameters.

Flexibility and Modularity: Because LoRA updates are applied through modular, low-rank matrices, they can easily be added or removed to switch tasks or domains. This modularity means that a single base model can be adapted to multiple tasks by loading only the specific low-rank adaptations required, allowing for rapid task-switching without having to reload or reconfigure the model completely.

By focusing only on a few low-rank matrices rather than the entire model, LoRA reduces the number of trainable parameters by orders of magnitude. This efficiency enables fine-tuning even on hardware with limited memory capacity, making it a scalable option for large models. Also, despite using fewer trainable parameters, LoRA often performs comparably to full fine-tuning. This is partly because the low-rank matrices focus on task-specific information without disrupting the underlying model architecture, which is already optimized for general linguistic and contextual knowledge. Thus, LoRA offers a highly efficient and flexible approach for fine-tuning large models, combining the performance benefits of full fine-tuning with the efficiency of parameter-efficient methods.

4. Theoretical Implications of LoRA: Discuss the theoretical implications of introducing low-rank adaptations to the parameter space of large language models. How does this affect the expressiveness and generalization capabilities of the model compared to standard fine-tuning?

1. Controlled Adaptation and Parameter Efficiency

LoRA's adaptation mechanism implies that large, over-parameterized language models may contain a significant amount of redundancy in their parameter space. Since only low-rank modifications are necessary for many tasks, it suggests that the model's general-purpose representations are broadly sufficient, and targeted, small adjustments are enough to achieve task-specific expressiveness.

2. Enhanced Generalization through Low-Rank Constraining

The low-rank matrices introduced by LoRA adapt model behavior without fundamentally altering the original pre-trained weights. As a result, the base model theoretically retains its general-purpose capabilities, and the low-rank adaptation can more flexibly transfer across related tasks or domains.

3. Theoretical Foundation for Model Modularity

LoRA’s modular low-rank adaptations align well with continual learning frameworks, where models must adapt to new tasks without forgetting previous ones. By only altering a small subset of parameters in a modular way, LoRA could theoretically enable continuous adaptation with reduced interference between tasks, as each task can have its unique low-rank modification without disrupting previous adaptations.

By limiting the adaptation to a low-rank space, LoRA acts as a form of regularization, potentially reducing the risk of overfitting that can occur when a model is fine-tuned on limited data. The restricted parameter space constrains the model’s updates, which can help it generalize better to out-of-sample data. Also, LoRA enables the model to capture task-specific features without altering the entire parameter space, indicating that these low-rank matrices are sufficient to encode the unique patterns needed for individual tasks. While this is powerful, it also means that LoRA may be less expressive for tasks requiring highly nuanced modifications that interact deeply with the model’s pre-trained weights.

Hyperparameters used to train the model(s)

loss function: Cross-entropy

number of epochs: 3

learning rate: 5e-5

optimizer: AdamW

Prompt Tuning

Results:

1) Evaluation Loss:

```
100%|██████████| 750/750 [02:02<00:00,  6.14it/s]
Evaluation Loss: 5.079626584370931
```

2) ROUGE Score:

```
100%|████████| 375/375 [25:35<00:00,  4.10s/it]
Soft Prompt Tuning Results: {'rouge1': 0.10171458743372636, 'rouge2': 0.01299608469665739, 'rougeL': 0.08439815559521
574, 'rougeLsum': 0.09592139365196647}
```

3) Number of parameters, training time, GPU compute

```
Total parameters: 124442112
Trainable parameters: 2304

Total training time: 2602.39 seconds
GPU Memory Usage after training: 1120.33 MB
```

LoRA Fine-Tuning

Results:

1) Evaluation Loss:

```
100%|██████████| 750/750 [01:58<00:00,  6.30it/s]
Evaluation Loss: 3.989332985242208
```

2) ROUGE Score:

```
100%|████████| 375/375 [01:32<00:00,  4.04it/s]
LoRA Results: {'rouge1': 0.2684574096981217, 'rouge2': 0.1288690725452124, 'rougeL': 0.17984825972260193, 'rougeLsu
m': 0.22527520434759374}
```

3) Number of parameters, training time, GPU compute

```
Total parameters: 125029632
Trainable parameters: 589824

Total training time: 2515.61 seconds
GPU Memory Usage after training: 2226.66 MB
```

Traditional Fine-Tuning

Results:

1) Evaluation Loss:

```
100%|██████████| 750/750 [01:54<00:00,  6.56it/s]
Evaluation Loss: 3.9608619702657064
```

2) ROUGE score:

```
100%|████████| 375/375 [01:27<00:00,  4.27it/s]
Traditional Tuning Results: {'rouge1': 0.2683619954165428, 'rouge2': 0.1289161370106166, 'rougeL': 0.1798940150849210
3, 'rougeLsum': 0.22527399322267508}
```

3) Number of parameters, training time, GPU compute

```
Total parameters: 124439808
Trainable parameters: 45685248

Total training time: 2679.52 seconds
GPU Memory Usage after training: 3232.51 MB
```

Analysis:

Higher evaluation loss and lower ROUGE score for prompt tuning suggests that the approach of updating only soft prompt embeddings while keeping the main model parameters frozen limits the model's ability to adapt fully to the task. LoRA Fine-tuning performs comparably to Traditional Fine-tuning across both evaluation loss and ROUGE scores. Given LoRA's lower memory footprint and efficiency, it could be a preferred approach if computational resources or memory constraints are considerations.

The number of trainable parameters and GPU usage is minimum for prompt tuning due to its isolated soft prompt updates. LoRA Fine-tuning requires moderate number of trainable parameters and GPU usage, efficiently updating specific matrices without modifying the bulk of the model. Traditional Fine-tuning incurs the highest resource demand, as it involves updating the model's parameters in the classifier layers.